Recursion and Backtracking

Problem 1: understanding Recursion mechanics

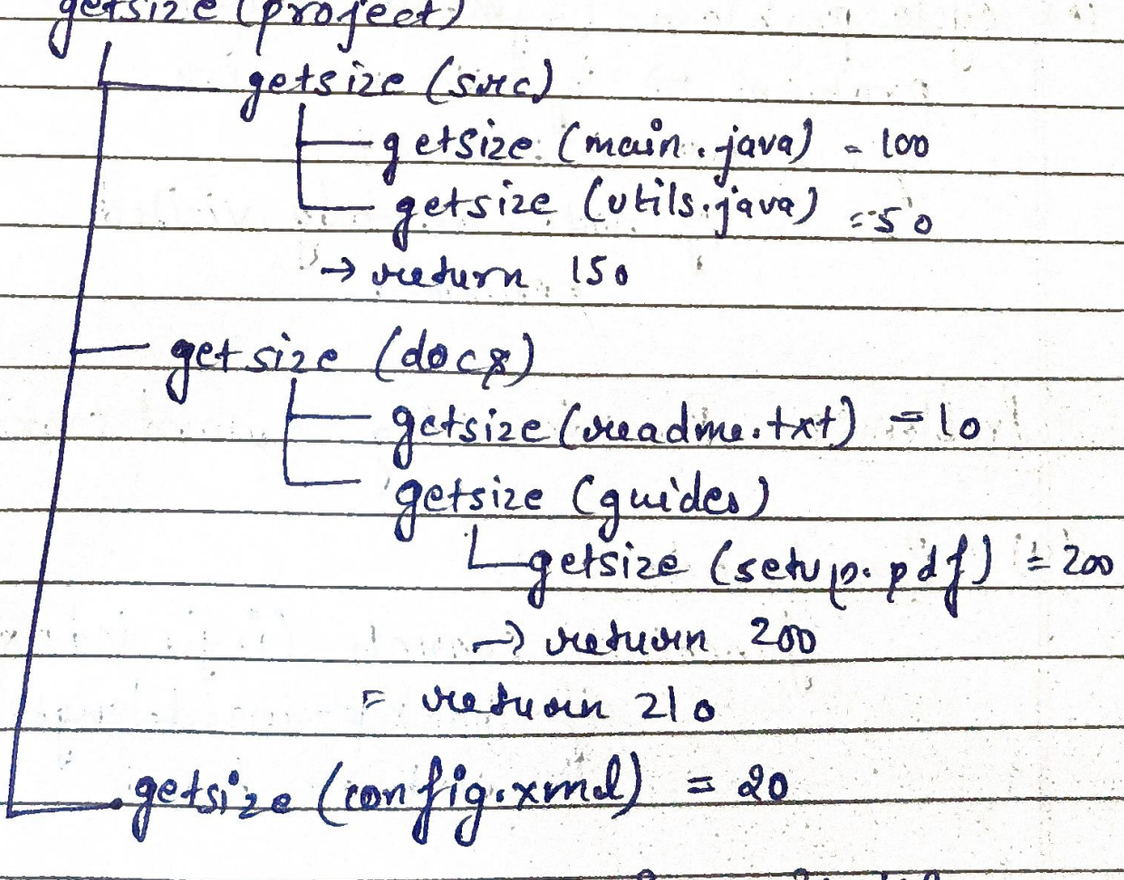
(a) Recursive Pseudo code.

```
function getsize(node):
    if node is FILE:
        return node.size
    total = 0
    for each child in node : //recursive case
        total += getsize(child)
    return total
```

Best case → when node is a file
Recursive case → when node is a directory

(b) Execution Trace (call stack):

stack expansion



150 + 210 + 20 = 300 KB

(c) Time complexity

each file + directory visited once

Time complexity = O(N)

where, N = total files + directories.

(d) Iterative solution is possible.

use: stack (DFS) or Queue (BFS)

```
stack.push (root)
while stack not Empty:
    node = stack.pop()
    if file → add size
       else → push children
```

Comparison:

Approach	Pros	cons
Recursion	simple, clean	stack overflow risk
Iterative	More control	slightly complex.

(e) cycle problem (Symbolic links)

problem → infinite recursion.

fix: use visited set

```
if node already visited
    return 0
```

Problem: 2 Backtracking (word search)

(a) Pseudocode

```
function search (i, j, index):
    if index == word.length;
        return true;
```

if out of bounds or visited or grid[i][j] != word[index]:
    return false

mark visited

for all 8 directions:
    if search(new-i, new-j, index+1):
        return true

unmark visited
return false

State : (i, j, index)
choices : 8 directions
constraint : can't reuse cell

(b) Decision Tree (simplified)

start from D(2,0)

D → R → E → A → M

D → E → wrong → backtrack.

(c) Execution flow

(i) Pick D
(ii) Try neighbours → find R
(iii) continue → E
(iv) continue → A
(v) continue → M

if mismatch : backtrack

(d) Time complexity : worst case = $O(N * M * 8^L)$

where N x M = grid
L = word length

(e) result = [ ]
if index == word.length:
add path to result.

Problem 3: N-Queens

(a) Search space
without optimization:
$N^N = 8^8 = 16,777,216$

one queen per row:
$N! = 8! = 40,320$

b) optimized solution
use 1D array → board[row] = col
Track:
columns [ ]
diag1[ ] → (row - col)
diag2[ ] → (row + col)

Pseudocode

```
function solve(row):
    if row == N:
        print solution
        return
    for col in 0 to N-1:
        if column[col] or diag1[row-col]
            or diag2[row+col]:
            continue
        place queen
        mark arrays
        solve(row+1)
        remove queen
        unmark array.
```

(c) Trace for N = 4

- Row 0 → col 1
  Row 1 → col 3
  Row 2 → col 0
  Row 3 → col 2 ✓

(d) if solution found:
    return true.

stop recursion early.

(e) if (row, col) is forbidden:
    continue

complexity: still exponential
slightly reduced search space.